

CMP_SC 4050: Design and Analysis of Algorithms I

Group Algorithm Engineering Assignment 1

Greedy Algorithms Under Pressure: Interval Scheduling

100 points | Group assignment

Assigned: 10/01/2026 Due: 10/09/2026 11:59 pm

Goal. In this assignment, your group will implement several plausible greedy algorithms for interval scheduling, experimentally evaluate them, automatically search for counterexamples, and compare their behavior with an exact solver.

implement → test → break → measure → explain

This assignment is intentionally different from the individual problem sets. The emphasis is on implementation, experimentation, adversarial testing, reproducibility, and interpretation. You are still responsible for explaining why your code and experiments support the claims you make.

Problem

You are given n jobs. Job j has a start time s_j and finish time f_j , with $s_j < f_j$.

Two jobs are compatible if they do not overlap. For this assignment, treat intervals as half-open:

$$[s_j, f_j)$$

Thus, a job finishing at time t is compatible with a job starting at time t .

Objective: select the maximum possible number of mutually compatible jobs.

AI Policy

AI-assisted assignment. Generative AI is permitted on this group engineering assignment. Your group may use AI tools for activities such as brainstorming, implementation assistance, debugging, test generation, or explanation. However, your group remains responsible for the correctness of every algorithm, experiment, claim, and line of submitted code.

Your submission must include a short AI Use Statement. Identify the tools used and briefly describe how they contributed. If your group did not use generative AI, simply state that.

AI output is not evidence of correctness. If an AI system proposes an algorithm, optimization, counterexample, explanation, or code fragment, your group must independently test and validate it.

Part 1: Implement Four Scheduling Strategies

25 pts

Implement all four algorithms below. Each implementation must return the selected set of jobs, not only the number selected.

A. Earliest Finish Time

Repeatedly choose the compatible job with the smallest finish time. This is the greedy algorithm studied in class.

B. Earliest Start Time

Repeatedly choose the compatible job with the smallest start time.

C. Shortest Duration

Repeatedly choose the compatible job minimizing

$$f_j - s_j$$

D. Your Group's Heuristic

Design one additional greedy rule of your own. It may use information such as latest start time, fewest conflicts, a combination of start/finish times, or another locally computed priority. You may not simply reproduce one of A-C under a different name.

For every heuristic, clearly specify:

- how jobs are ordered or selected;
- how ties are broken;
- its asymptotic running time.

Engineering requirement. Tie-breaking must be deterministic or explicitly randomized and documented. Your experiments should be reproducible.

Part 2: Build an Exact Solver

15 pts

For small instances, implement an algorithm that always finds an optimal solution. A brute-force algorithm is sufficient. For example, your algorithm may examine subsets of the jobs and determine which subsets are mutually compatible.

You are not expected to make this solver efficient for large n . Your report should state its asymptotic running time and explain why it is practical only for relatively small instances.

Ground truth. The exact solver acts as your experimental oracle. If it returns an optimal solution of size $\text{OPT}(I)$ for an instance I , you can objectively determine whether a greedy heuristic is optimal on that instance.

Part 3: Automatically Find Counterexamples

20 pts

Write a program that generates or searches through interval-scheduling instances and automatically finds examples where a greedy heuristic produces a suboptimal solution.

You must find at least:

- one counterexample for Earliest Start;
- one counterexample for Shortest Duration;
- one counterexample for your group's heuristic, if one exists in your search.

For every counterexample, report:

- the complete set of input intervals;
- the jobs selected by the heuristic;
- an optimal solution;
- the heuristic solution size;
- the optimal solution size.

For example:

$$|\text{Greedy}(I)| = 2, \quad \text{OPT}(I) = 4$$

Include a small visualization or timeline for at least one counterexample.

Important logical distinction. Finding a counterexample proves that an algorithm is not always optimal. Failing to find a counterexample does not prove that an algorithm is optimal. Your report must explicitly discuss this distinction.

Part 4: Test the Correct Greedy Algorithm
15 pts

Generate a large collection of small random instances for which your exact solver is still practical. For every instance I , compare

$$|\text{EarliestFinish}(I)|$$

with

$$\text{OPT}(I).$$

Your group must test at least 100 randomly generated instances. Use multiple types of generated instances rather than drawing every interval from exactly the same distribution.

For example, include several of the following:

- relatively sparse interval sets;
- heavily overlapping interval sets;
- instances with varied interval lengths;
- instances with clustered start times.

Report:

$$(\# \text{ instances where Earliest Finish matched OPT}) / (\# \text{ instances tested})$$

You should expect Earliest Finish to agree with the exact solver on every correctly implemented test instance. However, experimental agreement is evidence that your implementation is working. It is not a proof that the algorithm is optimal.

Part 5: Empirical Comparison of the Heuristics
10 pts

For your random instances, compare the solution quality of all four greedy strategies. For each heuristic H , compute

$$|H(I)| / \text{OPT}(I)$$

A value of 1 means the heuristic found an optimal solution. A value such as 0.75 means the heuristic scheduled 75% as many jobs as the optimum.

Report at least:

- the percentage of instances for which each heuristic was optimal;
- the average value of $|H(I)| / \text{OPT}(I)$;

- the worst instance you observed for each heuristic.

Present the results using a table or graph. Then discuss what you observe. Do not simply provide numbers: explain what kinds of instances seem to mislead each heuristic.

Part 6: Runtime Experiment

10 pts

The exact solver will quickly become impractical. For this part, benchmark only the greedy algorithms.

Generate increasingly large instances. A reasonable starting set is:

$$n = 100, 1,000, 10,000, 100,000$$

Measure the running time of each greedy algorithm. Run each experiment multiple times and report a representative value, such as the median.

Compare the empirical measurements with your theoretical runtime analysis. In particular, explain where sorting appears in the running time.

Do not overclaim. A few timing measurements do not prove an asymptotic bound. Discuss whether the measurements are consistent with your theoretical analysis.

Part 7: Engineering Reflection

5 pts

Include a short reflection answering all of the following:

1. Which incorrect greedy heuristic appeared most convincing before you tested it?
2. What did the counterexample reveal about why the local rule fails?
3. What did the exact solver contribute to the development process?
4. What is the difference between experimentally validating an algorithm and proving it correct?

This section may be relatively short, but it should contain actual analysis rather than generic statements.

Required Sanity Tests

Before running large experiments, verify that your framework detects these known failures. These tests are intended to catch implementation mistakes early.

Sanity Test 1: Earliest Start Fails

Intervals:

```
A = (0, 10)
B = (1, 2)
C = (2, 3)
D = (3, 4)
E = (4, 5)
```

Earliest Start selects A, giving 1 job. An optimal solution selects B, C, D, E, giving 4 jobs.

Sanity Test 2: Shortest Duration Fails

Intervals:

```
A = (2, 4)
B = (0, 3)
C = (3, 6)
```

Shortest Duration selects A, giving 1 job. An optimal solution selects B and C, giving 2 jobs.

After these pass. Use automated generation/search rather than manually constructing the rest of your experimental evidence.

Required Deliverables

Submit one package per group containing all of the following.

1. Source Code

- all four greedy algorithms;
- the exact solver;
- test-instance generator(s);
- counterexample search;
- experimental scripts.

2. README

- language and version;
- how to run the program;
- expected input format;
- how to reproduce the experiments.

3. Engineering Report

Approximately 4-6 pages, including algorithm descriptions, counterexamples, experimental methodology, tables/plots, discussion, and conclusions.

4. Machine-Readable Experimental Results

Submit CSV, JSON, or an equivalent format containing enough information to reproduce the figures and summary statistics in your report.

5. Group Contribution Statement

Briefly state the primary contribution of each team member.

6. AI Use Statement

Briefly identify any generative AI tools used and how they contributed. If none were used, state that explicitly.

Grading Rubric

Component	Points
Four greedy implementations	25
Exact solver	15
Counterexample search	20
Validation of Earliest Finish	15
Empirical comparison	10
Runtime experiment	10
Engineering reflection	5
Total	100

Code will be evaluated primarily on correctness, reproducibility, and engineering quality, not cleverness. A straightforward brute-force exact solver is completely acceptable.

Start early. Do not wait until the final two days to build the exact solver and experimental infrastructure. Your experiments should be reproducible from code rather than constructed manually for the report.